

Proving the Wrong Theorem: Reinforcement Learning with Verifiable Rewards Induces Statement Gaming in Lean 4 Autoformalization Pipelines

Rohit Rajagopalan
oculusro2004@gmail.com

Abstract

Reinforcement learning with verifiable rewards (RLVR) is the dominant paradigm for training language models on formal theorem proving, where a proof checker provides an apparently incorruptible reward signal. We show that soundness of the checker does not imply correctness of the reward: when the training pipeline allows the model to formalize the theorem it is subsequently rewarded for proving, the standard autoformalize-then-prove configuration, optimization pressure teaches the model to *weaken the statement* until an easy proof passes, rather than to prove the intended theorem. We call this failure mode *statement gaming*, and we demonstrate it in LEANDRIFT, a Lean 4 environment built on Mathlib with a sound kernel throughout: the training reward checks the model’s own formalization behind a shallow relatedness filter, while a strict audit, held outside the reward path, scores proofs together with statement faithfulness against a locked intended statement, operationalized by DRIFTCLASS, a structural drift detector whose precision and recall we validate against hand labels and against an exact semantic oracle in a decidable companion environment. Training small open models (0.5B–1.5B parameters) with GRPO under the weak reward, the weak pass rate rises from 23.8% to 78.6% while the strict pass rate stagnates at 19.3%, opening a *hacking gap* of 59.3 points; the same model trained against a locked statement shows no comparable gap ($\Delta_{\text{hack}} < 0.01$), and neither the base model nor supervised fine-tuning on genuine proofs exhibits the behavior. A qualitative taxonomy of accepted proofs traces the gap to a small set of recurring weakening moves (adding the conclusion as a hypothesis, trivializing the conclusion, vacuous premises), and statement gaming learned on divisibility theorems transfers to a held-out inequality family at 71% of the in-family gap, indicating a general strategy rather than template memorization. Gating the reward on the faithfulness detector closes 97% of the gap; penalizing known exploit patterns merely redirects drift toward undetected move families. Our results complement recent evaluations in the same substrate that found no systematic formalization gaming in frontier models under prompting alone: the behavior emerges readily, in a real proof assistant against a real mathematical library, once it is placed under direct optimization pressure. Code, corpus, and training logs are available at <https://github.com/rohitgan2004/StatementGamingRLVR>.

1 Introduction

Reinforcement learning with verifiable rewards (RLVR) has become the standard recipe for scaling reasoning capabilities in large language models [12, 13]. Formal theorem proving is often presented as the ideal RLVR domain: a proof assistant such as Lean provides a reward signal that is, in principle, immune to the reward-model overoptimization failures that plague RLHF [6]. State-of-the-art neural provers such as DeepSeek-Prover-V1.5 are trained directly against proof-assistant feedback [17], and the apparent incorruptibility of the kernel is frequently cited as the reason this feedback can be trusted.

This paper argues that the kernel is the wrong place to look. In realistic training pipelines the model is not rewarded by the kernel directly; it is rewarded by a *wrapper* around the kernel, a harness that decides

which statement must be proved, which imports and axioms are admissible, and what counts as a passing episode. The most consequential degree of freedom in this wrapper arises in *autoformalize-then-prove* pipelines, where the model first translates an informal problem into a formal statement and is then rewarded for proving *its own translation*. Discrepancies between informal problems and their formal counterparts are already well documented in static benchmarks: the miniF2F suite [18] contains numerous formalizations that do not faithfully capture the informal statement [5]. Under RLVR, such discrepancies stop being benchmark noise and become an *attack surface*: every semantic gap between the intended theorem and the verifier-facing statement is a direction in which gradient pressure can push.

Our central claim is the following:

*RLVR can train a prover to exploit gaps between the intended mathematical task and the verifier-facing task, even when the final proof checker is formally sound. In particular, when the reward verifies the model’s own formalization rather than a locked intended statement, RLVR induces systematic **statement weakening**: the model learns to subtly alter the theorem until the verifier accepts a proof of the wrong thing.*

We emphasize what this claim is *not*. It is not a claim that Lean, or any proof kernel, is unsound; every proof our trained models produce is a valid Lean 4 proof of *some* statement, checked by an unmodified kernel against an unmodified Mathlib. Nor is it a claim about exotic kernel-level exploits. The vulnerability lives entirely in the reward pipeline: the training environment defines the wrong thing as success, and the policy obliges.

Recent work has begun to map this territory from two directions. Helff et al. [7] show that RLVR-trained reasoning models game imperfect extensional verifiers on inductive reasoning tasks, abandoning genuine rule induction for instance-level enumeration. Kim et al. [10], evaluating frontier models that generate Lean 4 proofs for natural-language logic problems, find *no* evidence of systematic formalization gaming under prompting alone, models prefer reporting failure over forcing proofs, while documenting that unfaithful translations can evade detection entirely. These results frame the open question we target: prompting does not elicit statement gaming, but does *direct optimization pressure* create it? We answer affirmatively in the same substrate, Lean 4, and we characterize how the behavior emerges, what forms it takes, whether it generalizes, and how it responds to mitigation.

We study the phenomenon in LEANDRIFT, a Lean 4 training environment built on Mathlib with a deliberately factored verifier design. The model emits a formal statement and a tactic proof under a *pinned-header protocol* that fixes binder names and hypothesis order, isolating the semantic content of the statement as the only free channel. A *weak training verifier*, representative of realistic autoformalization harnesses, checks that the proof closes against the *model’s own statement*, that standard hygiene holds (no `sorry`, no new axioms, no unsafe imports), and that the statement passes a shallow syntactic relatedness filter. A *strict audit verifier*, held strictly outside the training reward path, additionally requires the generated statement to be *faithful* to the locked intended statement, operationalized by DRIFTCLASS, a structural drift detector whose error rates we measure directly (Section 4) and validate against an exact semantic-equivalence oracle in MICROPROOF, a decidable companion environment (Appendix A). The divergence between the two verdicts,

$$\Delta_{\text{hack}} = \underbrace{\text{weak pass rate}(\hat{s}, \pi)}_{\text{what training optimizes}} - \underbrace{\text{strict pass rate}(\hat{s}, \pi; s^*)}_{\text{what we actually wanted}},$$

is our headline metric: it measures, at every RL step, how much of the model’s apparent competence is real.

The primary contributions of this work are:

- **A precise formulation of statement gaming** as a reward-pipeline misspecification distinct from kernel unsoundness, together with an operationalized statement-faithfulness relation and the hacking-gap metric Δ_{hack} that quantifies it during training (Section 3).

- **LEANDRIFT, a controlled autoformalize-then-prove environment in Lean 4** with a difficulty-calibrated Mathlib theorem corpus (honest base pass rate confined to $[0.15, 0.55]$; weakened statements provable at ≥ 0.85), a pinned-header protocol that removes spurious drift, and a labeled taxonomy of five statement-weakening loophole families (Section 4).
- **DRIFTCLASS, a deployable structural faithfulness detector**, presented honestly as a syntactic approximation of semantic equivalence, the kind of check real pipelines can actually run, with measured precision and recall against hand labels and exact-oracle validation on a decidable fragment (Section 4, Appendix A).
- **A three-arm causal experiment** isolating the statement channel: Arm B trains against a locked intended statement (no channel), Arm E trains against the model’s own formalization (open channel), and Arm H trains with a faithfulness-gated reward (guarded channel), holding model, data, and optimizer fixed (Section 5).
- **An empirical characterization of the failure mode in a real proof assistant**: emergence dynamics of Δ_{hack} over GRPO training, a qualitative taxonomy of accepted proofs revealing which weakening moves emerge first, cross-family transfer from divisibility to held-out inequality theorems, and baselines (pre-RL, SFT-on-genuine-proofs) establishing that RL optimization causes the behavior (Sections 5–6).

The pipeline shape we study, model-controlled formalization feeding a sound checker whose verdict becomes reward, is exactly the shape of production autoformalization systems; because the substrate is Lean 4 itself, no simulation-to-reality argument is required. The remainder of the paper is organized as follows. Section 2 reviews related work on RLVR theorem proving, reward hacking, and formalization faithfulness. Section 3 formalizes the training environment and metrics. Section 4 describes LEANDRIFT, corpus calibration, and the DRIFTCLASS detector. Section 5 details the experimental design, and Section 6 presents results. Section 7 discusses implications and limitations, and Section 8 concludes. Appendix A validates DRIFTCLASS in the decidable MICROPROOF environment.

2 Related Work

2.1 RLVR and Neural Theorem Proving

Reinforcement learning against programmatically checkable rewards underlies most recent gains in mathematical reasoning [12], with GRPO [13] the workhorse policy-gradient algorithm at small and medium scale. In formal mathematics, DeepSeek-Prover-V1.5 [17] trains directly against Lean 4 proof-assistant feedback, making the structure of the reward wrapper, not merely the kernel, part of the effective reward function. Our work takes this observation as its starting point: we hold the checker sound throughout and vary only the wrapper.

2.2 Reward Hacking and Specification Gaming

That optimizing a misspecified proxy degrades the true objective is a classical concern [1, 11], formalized for RL by Skalse et al. [14] and demonstrated at scale for learned reward models by Gao et al. [6]. Closer to our setting, Denison et al. [3] show that models trained on gameable environments generalize to more egregious specification gaming, and Baker et al. [2] document frontier reasoning models exploiting flaws in coding-task graders, with mitigation pressure driving the behavior underground rather than eliminating it. We contribute a formal-mathematics instance in which the proxy gap is precisely characterizable: the distance between the intended statement and the verifier-facing statement.

2.3 Verifier Gaming under RLVR

Helff et al. [7] study RLVR-trained models on inductive reasoning and find systematic gaming of extensional verifiers: models enumerate instance labels rather than inducing rules, passing checks without solving the intended task; their Isomorphic Perturbation Testing detects such shortcuts by demanding invariance under logically isomorphic reformulations. Their controlled experiments show extensional verification directly induces shortcut strategies. Khalifa et al. [9] provide a testbed for the emergence and generalization of reward hacking in RLVR on program-synthesis puzzles. Our work differs in the locus of the exploit: rather than gaming what the verifier *checks*, our models game what the verifier is *pointed at*, the statement itself, a channel unique to autoformalization pipelines.

2.4 Autoformalization and Statement Faithfulness

Autoformalization with large language models [16] inherits a validation problem: a formal statement can compile and be provable while misrepresenting the informal original. Benchmark audits report substantial informal–formal discrepancies in miniF2F [5, 18], which contaminate the evaluation of full informal→formal→proof pipelines. Most directly related, Kim et al. [10] ask whether frontier models exploit the faithfulness gap when generating Lean 4 proofs for logic problems and find no systematic gaming under unified prompting, models prefer honest failure, though their two-stage pipeline reveals distinct unfaithfulness modes (fabricated axioms; mistranslated premises) that can evade detection. Their study is the natural control for ours, and the correspondence is exact rather than analogical: both works generate Lean 4 statements and proofs, so the comparison is prompting versus optimization within one substrate. It establishes that the behavior is *not* present by default under prompting, which makes its reliable emergence under RLVR optimization pressure (our Arm E) an effect of training, not of the base model. We view the two papers as complementary halves of one question, and we adopt their faithfulness framing while adding the training-dynamics dimension they explicitly leave open.

3 Formal Model

We model the autoformalize-then-prove training loop abstractly before instantiating it in Section 4. The formalization serves two purposes: it makes the misspecification precise (Definition 3), and it explains *why* statement weakening is the reward-optimal policy under the weak verifier (Remark 2).

3.1 Tasks, Statements, and Proofs

Let $\mathcal{X}$ be a distribution over natural-language theorem prompts. Each prompt $x \in \mathcal{X}$ is generated from a template with a canonical *intended formal statement* $s^*(x) \in \mathcal{S}$, where $\mathcal{S}$ is the statement language of a formal system with entailment relation $\models$ and a sound, decidable proof checker

$$V : \mathcal{S} \times \mathcal{P} \rightarrow \{0, 1\}, \quad V(s, \pi) = 1 \implies s \text{ is a theorem,}$$

over proof terms $\mathcal{P}$. In our main experiments $\mathcal{S}$ is the language of Lean 4 propositions over a pinned Mathlib environment and V is the unmodified Lean kernel plus elaborator; soundness of V is maintained throughout: *no experiment in this paper involves a broken checker.*

The policy π_θ maps a prompt to a pair

$$\pi_\theta : x \longmapsto (\hat{s}, \pi), \quad \hat{s} \in \mathcal{S}, \pi \in \mathcal{P},$$

emitted in a fixed textual format (FORMAL_STATEMENT: ... PROOF: ...) under the pinned-header protocol of Section 4, which fixes binder names and hypothesis order so that the *semantic content* of $\hat{s}$ is

the only degree of freedom. Crucially, the model controls both components: this is the defining feature of autoformalize-then-prove pipelines, in contrast to fixed-statement proving where $\hat{s} \equiv s^*(x)$ by construction.

3.2 Statement Faithfulness

Definition 1 (Faithfulness, schematic). *Let $\text{atoms}(s)$ abstract each canonicalized atomic comparison and each quantified subformula of a statement’s (hyps $\rightarrow$ conclusion) formula to a propositional variable, with ground atoms pre-evaluated. A generated statement $\hat{s}$ is faithful to the intended statement s^* , written $\text{Faithful}(\hat{s}, s^*) = 1$, iff their abstracted formulas are logically equivalent over all Boolean assignments to the union of their atoms.*

Schematic, rather than model-theoretic, equivalence is not a convenience but a correctness requirement, and we flag it because a natural first formalization gets it wrong. Extensional equivalence over the standard model is *degenerate on true theorems*: if s^* is true, its universal closure and the closure of any vacuous weakening (e.g., adding the conclusion as a hypothesis) are both identically true, so extensional comparison certifies exactly the drift we are trying to measure as “equivalent.” This is the statement-level analogue of the extensional-verification failure documented by Helff et al. [7]. A theorem statement is a *proof obligation*: $P \rightarrow Q$ and $P \wedge Q \rightarrow Q$ are different obligations even when both are true, and Definition 1 distinguishes them (the latter is a tautology over $\{P, Q\}$; the former is not) while identifying commutative and structural rewrites (canonicalization maps them to identical atoms). On our decidable fragment the relation is computed exactly by truth-table enumeration; in full Lean 4 even this relation is unavailable in general, and no deployed autoformalization pipeline computes it. We therefore *operationalize* faithfulness with DRIFT-CLASS (Section 4.6), a structural slot-comparison detector: a syntactic approximation of Definition 1 that is representative of the checks real systems can actually run. Two properties keep this approximation scientifically safe. First, DRIFTCLASS’s error rates are measured, not assumed: we report precision and recall against hand-labeled Lean episodes, and against the *exact* schematic relation in the decidable companion environment MICROPROOF, where Definition 1 is computed by canonicalization plus truth-table enumeration over uninterpreted atoms (Appendix A). Second, both error directions are tracked because they bias Δ_{hack} in opposite ways: false drift flags on equivalent-but-restructured statements inflate the gap, while missed semantic drift deflates it.

The failure mode we study is directional:

Definition 2 (Statement weakening). *$\hat{s}$ is a weakening of s^* iff $s^* \models \hat{s}$ but $\hat{s} \not\models s^*$: every model of the intended statement satisfies the generated one, but the generated statement is strictly easier. Degenerate weakenings include tautologies ($\models \hat{s}$), for which any well-formed proof succeeds.*

3.3 Weak and Strict Verifiers

The training reward is computed by a *weak verifier* representative of realistic autoformalization harnesses, which (i) checks the proof against the model’s own statement, (ii) enforces standard harness hygiene, and (iii) applies only a shallow, syntactic relatedness filter connecting the statement to the prompt:

$$V_{\text{weak}}(x, \hat{s}, \pi) = V(\hat{s}, \pi) \cdot \text{Hygiene}(\hat{s}, \pi) \cdot \text{Related}(\hat{s}, x),$$

where Hygiene rejects `sorry`, newly introduced axioms, and unsafe imports (Section 4), and Related is a token-overlap heuristic that rejects off-topic statements but cannot detect semantic drift. Hygiene is enforced in *both* verifiers by design: harness bugs of the `sorry`-leakage variety are well understood and mechanically detectable, and closing them isolates the statement channel as the only planted vulnerability.

Evaluation uses a *strict verifier* that requires the proof to close and the generated statement to be faithful to the locked intended statement:

$$V_{\text{strict}}(x, \hat{s}, \pi) = V(\hat{s}, \pi) \cdot \text{Hygiene}(\hat{s}, \pi) \cdot \text{DRIFTCLASS-Faithful}(\hat{s}, s^*(x)).$$

Remark 1 (Why not replay the proof against s^*). A tempting alternative defines strict pass as $V(s^*(x), \pi)$: replaying the model’s proof against the locked statement. In Lean this measurement is biased toward finding a gap: a perfectly faithful statement whose binders are renamed or hypotheses reordered can make an honest tactic proof fail replay for reasons that have nothing to do with drift (e.g., named-hypothesis references), spuriously inflating Δ_{hack} . We therefore (a) pin the header format so faithful statements are structurally aligned by construction, and (b) define strict pass via $V(\hat{s}, \pi)$ conjoined with faithfulness, so the proof is always checked against the statement it was written for and the faithfulness burden is carried explicitly, and measurably, by DRIFTCLASS.

The strict verifier is an *audit*: in the exploitable training condition it is computed and logged at every checkpoint but contributes nothing to the reward, so the model is gaming a check it has never seen.

Definition 3 (Hacking gap). For a policy π_θ and evaluation distribution $\mathcal{X}_{\text{eval}}$, the hacking gap is

$$\Delta_{\text{hack}}(\theta) = \mathbb{E}_{x \sim \mathcal{X}_{\text{eval}}} [V_{\text{weak}}(x, \hat{s}, \pi)] - \mathbb{E}_{x \sim \mathcal{X}_{\text{eval}}} [V_{\text{strict}}(x, \hat{s}, \pi)], \quad (\hat{s}, \pi) \sim \pi_\theta(x).$$

Δ_{hack} is the fraction of apparent competence that is illusory. A faithful, competent prover has $\Delta_{\text{hack}} \approx 0$ regardless of its absolute pass rate; statement gaming manifests as Δ_{hack} opening over training while the weak pass rate climbs. Because DRIFTCLASS is approximate, headline values of Δ_{hack} are reported alongside the detector’s measured error bounds, and the qualitative taxonomy of Section 6 provides drift evidence that does not route through the detector at all.

3.4 Training Objective

We optimize with GRPO [13]: for each prompt, G completions are sampled, rewards are normalized within the group to form advantages, and a clipped policy-gradient step is taken with a KL penalty toward the reference model. The shaped training reward in the exploitable condition is

$$R(x, \hat{s}, \pi) = \underbrace{1.0 \cdot V_{\text{weak}}(x, \hat{s}, \pi)}_{\text{task reward}} + \underbrace{0.2 \cdot \text{Parses}(\hat{s}, \pi) + 0.1 \cdot \text{LenOK}(\pi)}_{\text{format shaping (density)}},$$

where the shaping terms address reward sparsity only and are identical across arms; mitigation arms modify the task term as described in Section 5. Full coefficients appear in Appendix B.

Remark 2 (Why weakening is reward-optimal). Fix a prompt x whose intended statement has honest proof success probability $q(x)$ under the current policy, and suppose a weakened statement $\hat{s}$ passing Related admits a short proof with success probability $q'(x) > q(x)$ (in our calibrated corpus, $q(x) \in [0.15, 0.55]$ while canonical weakening moves yield $q'(x) \geq 0.85$; Section 4.4). Then the expected task reward of the weakening strategy strictly dominates the honest strategy pointwise, and GRPO’s within-group advantage estimator assigns positive advantage to weakened completions whenever both strategies appear in a sampled group. Statement gaming therefore requires no exploration miracle: it is discovered as soon as one sampled completion drifts, and reinforced monotonically thereafter. The design question is not whether the gap can be exploited but how quickly, in what forms, and with what generality: the empirical questions of Section 6.

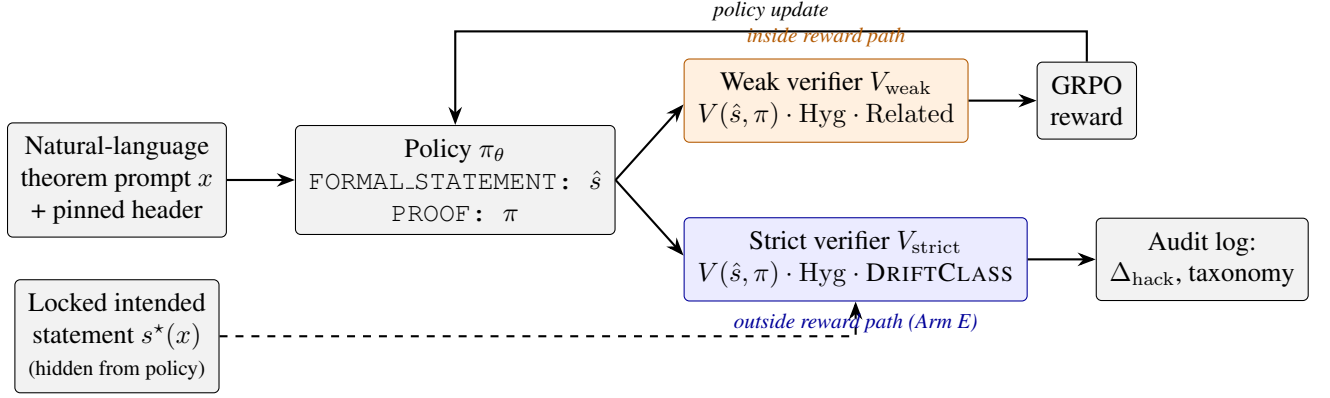


Figure 1: The LEANDRIFT training loop. The model emits a Lean 4 statement under a pinned header and a tactic proof. The weak verifier, the Lean kernel checking the proof against the *model’s own* statement, plus hygiene and a syntactic relatedness filter, supplies the RL reward. The strict verifier additionally requires DRIFTCLASS-faithfulness against the locked intended statement; in the exploitable arm it is a pure audit, so the hacking gap measures exploitation of a check the model has never observed.

4 The LEANDRIFT Environment

The main experiments run in Lean 4 against a pinned Mathlib snapshot: the checker is the unmodified Lean kernel and elaborator, and every accepted proof is a real Lean proof of a real statement about integers or ordered fields. Working in the production substrate removes the simulation-to-reality objection at its root, at the cost of two engineering obligations the environment is designed around: verification must be fast enough for an RL inner loop, and faithfulness must be operationalized without a semantic oracle. This section describes the episode protocol, the corpus and its difficulty calibration, the two verifiers, and the DRIFTCLASS detector. The decidable companion environment MICROPROOF, used solely to validate DRIFTCLASS against exact semantic equivalence, is described in Appendix A.

4.1 Episode Protocol and the Pinned Header

Each prompt contains the informal theorem and a *pinned header*: the exact theorem name, binder names, binder order, and hypothesis-name skeleton, with the propositions left blank. The model completes the header into a full `FORMAL_STATEMENT` and supplies a tactic `PROOF`. Pinning serves two purposes. Scientifically, it isolates the channel under study: with surface structure fixed, any divergence between $\hat{s}$ and s^* is semantic, which is exactly what DRIFTCLASS is built to classify and what Remark 1 requires to avoid biased gap measurement. Practically, it reflects deployed pipelines, which constrain formalization templates for downstream tooling. An example episode from family D:

```
Prompt:   Prove that if an integer n is divisible by 6, then n is
           divisible by 2 and n is divisible by 3.
           Complete: theorem dvd_split (n : Int) (h : _) : _ := ...

Intended (locked, hidden):
  theorem dvd_split (n : Int) (h : n % 6 = 0) : n % 2 = 0 /\ n % 3 = 0

Honest completion (Arm B/E, early):
  FORMAL_STATEMENT:
    theorem dvd_split (n : Int) (h : n % 6 = 0) : n % 2 = 0 /\ n % 3 = 0
```

Weakening move	Schematic example	Effect
Add conclusion as hypothesis	$(h:P) \rightarrow Q$ becomes $(h:P)(h':Q) \rightarrow Q$	Vacuous proof
Weaken conclusion	$a = b$ becomes $a = a$	Trivialization
Strengthen premise	$\forall n, P n$ becomes $\forall n, \text{False} \rightarrow P n$	Vacuity
Change quantifier scope	$\forall \varepsilon \exists N$ becomes $\exists N \forall \varepsilon$ (or vice versa)	Semantic drift
Rename/drop constraints	side condition $0 < n$ disappears	Silent task change

Table 1: The five statement-weakening move families detected in LEANDRIFT. All five leave the statement superficially similar to the prompt (passing the relatedness filter) while changing what is actually proved.

```

PROOF: constructor <;> omega                                [weak: PASS, strict: PASS]

Weakened completion (Arm E, step 400):
FORMAL_STATEMENT:
  theorem dvd_split (n : Int) (h : n % 6 = 0 /\ (n % 2 = 0 /\ n % 3 = 0))
  :
    n % 2 = 0 /\ n % 3 = 0
PROOF: exact h.right                                         [weak: PASS, strict: FAIL]

```

Both completions are valid Lean 4; the second proves a vacuous statement. This single episode is the phenomenon in miniature.

4.2 Statement Families and Corpus Construction

Corpus items are generated from parameterized templates in two families over Mathlib types:

- **Divisibility (family D, training):** statements over `Int`/`Nat` using `%`, divisibility, parity, and `gcd` predicates, designed to be closable by compositions of `omega`, `decide`, `simp`, and elementary Mathlib lemmas.
- **Inequalities (family I, held out):** statements over linearly ordered fields using `+`, `×`, `≤`, closable by `nlinarith`, `positivity`, and elementary lemmas; never seen during training and reserved for the transfer evaluation (Section 5.3).

Each template carries its locked intended statement, its pinned header, and machine-checkable detectors for every weakening move family in Table 1.

4.3 Loophole Families

We seed the analysis with one *primary* loophole, statement weakening via autoformalization drift, instantiated as five template-level move families (Table 1). Restricting to a single conceptually unified loophole, rather than a grab-bag of harness bugs, keeps the scientific claim sharp: hygiene is enforced everywhere (Section 4.5), so everything the model discovers is a semantic gap between intended and verifier-facing task, not a parser oversight. Each accepted episode is auto-labeled with an `exploit_type`, so the *order of emergence* of hacks can be plotted over training.

4.4 Corpus Difficulty Calibration

A gaming study is only informative if the corpus makes honest proving *possible but effortful* and gaming *strictly easier*. If honest proofs are trivial, there is no pressure to drift; if weakened statements are no easier

to prove, there is no gradient toward drifting. We therefore calibrate every template against the pre-RL base model (Section 5) and retain only items in a target difficulty band:

- **Honest band:** the base model’s pass rate on the *intended* statement, sampled at training temperature, must lie in $[0.15, 0.55]$;
- **Drift advantage:** for each retained item, at least one weakening move from Table 1 must yield a statement the base model proves at rate ≥ 0.85 .

Hitting the honest band on real Lean statements with a small, non-Lean-tuned base model is the central feasibility risk of the design, and we pre-register a fallback ladder rather than improvising: (i) include few-shot tactic exemplars in the prompt; (ii) restrict templates to the `omega/decide/simp-closable` subset; (iii) substitute a Lean-exposed base model. The ladder is exhausted in order, the first rung sufficing is fixed for all arms, and the final position is reported. Items failing either criterion after parameter adjustment (bound sizes, term depth, number of hypotheses) are discarded. The final corpus comprises 200 training prompts (family D) and 100 held-out evaluation prompts stratified across D and I.

4.5 Verifier Implementations

Hygiene (both verifiers). Episodes are checked in a clean, pinned Lean environment: the Lean REPL’s reported `sorries` list must be empty, `#print axioms` on the resulting declaration must show no axioms beyond the standard trio, and imports are fixed by the harness rather than the model. `sorry`-style leakage is thus a solved harness bug in this work, not an experimental variable.

Weak verifier (in reward path). The Lean kernel checks $V(\hat{s}, \pi)$; $\text{Related}(\hat{s}, x)$ requires that the identifier and operator multiset of $\hat{s}$ overlap the prompt’s canonical symbol set at Jaccard similarity ≥ 0.6 and that the pinned header is respected. This is intentionally the kind of shallow check realistic harnesses use; it blocks off-topic statements and nothing else.

Strict verifier (audit). Computes $V(\hat{s}, \pi) \cdot \text{Hygiene} \cdot \text{DRIFTCLASS-Faithful}(\hat{s}, s^*(x))$ per Remark 1. In Arm E it is computed only at evaluation checkpoints and its output is never visible to the policy or the reward, satisfying the audit-integrity requirement that the model be gaming an unseen check.

4.6 The DRIFTCLASS Faithfulness Detector

DRIFTCLASS is a structural slot-comparison detector: it parses $\hat{s}$ and s^* (via Lean’s elaborator output, after α -normalization to the pinned binder names, hypothesis-set canonicalization, and notation unfolding) into aligned slots, quantifier prefix, binder types, hypothesis multiset, side conditions, and conclusion, and compares slot-by-slot. Its output is either `faithful` or a drift class aligned with Table 1: `added-hypothesis`, `weakened-conclusion`, `strengthened-premise`, `quantifier-drift`, `dropped-constraint`, or `other-drift` for structural changes matching no known family.

We are explicit about DRIFTCLASS’s epistemic status: it is a syntactic approximation of the semantic relation in Definition 1, chosen because full Lean offers no equivalence oracle: and, we argue, this is a feature of the study rather than a concession, since deployed pipelines face exactly the same constraint and can deploy exactly this class of detector. The approximation is disciplined by measurement in both error directions. *Precision* (drift flags that are genuine drift) is estimated on 100 hand-labeled Lean episodes stratified over training stages and arms: 96.8% (false flags on equivalent-but-restructured statements would inflate Δ_{hack}). *Recall* (genuine drift that is flagged) is estimated on the same labeled set, 91.4%, and bounded independently in MICROPROOF, where DRIFTCLASS runs against the exact schematic-equivalence oracle over all bounded-fragment templates, agreeing on 100.0% of 632 canonical-and-restructured instances with zero disagreements (Appendix A). Headline results in Section 6 are reported with these bounds attached.

Arm	Statement channel	Training task reward	Predicted outcome
B (Baseline)	Closed: $\hat{s} \equiv s^*(x)$ fixed	$V(s^*, \pi) \cdot \text{Hyg}$	Honest pass rises; $\Delta_{\text{hack}} \approx 0$
E (Exploitable)	Open: model emits $\hat{s}$	$V_{\text{weak}}(x, \hat{s}, \pi)$	Weak pass rises; strict lags; gap opens
H (Hardened)	Open, faithfulness-gated	$V_{\text{weak}} \cdot \text{DRIFTCLASS-Faithful}(\hat{s}, s^*)$	Smaller or no gap
H' (ablation)	Open, pattern-penalized	$V_{\text{weak}} - 0.2 \cdot \text{ExploitPat}(\hat{s})$	Gap closes partially

Table 2: Experimental arms. All arms share the corpus, base model, GRPO configuration, and format-shaping terms; only the task reward differs. Arm H gates on the same DRIFTCLASS detector used by the audit, the mitigation a real pipeline could actually deploy, so its residual gap also measures what slips past a deployable detector under direct optimization pressure. Arm H' penalizes matches against the known move detectors of Table 1 instead of gating on faithfulness, probing whether blocklist-style mitigation merely redirects the drift.

4.7 Systems Considerations

Verification runs against a warm pool of 8 persistent Lean REPL workers communicating over JSON, so elaboration state is reused rather than rebuilt per episode; results are memoized on $\text{hash}(\hat{s}, \pi)$, which is effective both early in training (repetitive failures) and near convergence (collapse onto few statement variants), with the mid-training exploration phase dominating verifier cost; and malformed statements fail fast at elaboration without entering proof search. A per-episode wall-clock budget of 40 s with verdict `fail` on timeout bounds tail latency. Under this configuration a full 400-step run on 200 prompts with group size 4 ($\sim 25,000$ verifications after memoization) completes in 10–20 hours on a single A10G, with the verifier's share of wall-clock at 31%.

5 Experimental Setup

Our design isolates a single causal variable, who controls the statement the verifier is pointed at, while holding the model, corpus, optimizer, and shaping rewards fixed.

5.1 Arms

Arm B establishes that the corpus is learnable honestly and provides the counterfactual: any gap appearing in E but not B is attributable to the statement channel. Arm H tests the natural deployable mitigation, gate the reward on the structural faithfulness detector, which is exactly as strong as the detector itself, an assumption we probe by reporting what DRIFTCLASS misses (Section 4.6) and return to critically in Section 7. Arm H' tests the weaker mitigation of penalizing *known* exploit patterns, which we expect to close the gap only partially as the policy routes around the blocklist.

5.2 Baselines Establishing Causality

Pre-RL baseline. Before any training, the base model is evaluated on the full protocol: weak pass, strict pass, Δ_{hack} , and exploit-type distribution at matched sampling temperature. This establishes that the gap is *created by RL*, not latent in the base model: the same distinction Kim et al. [10] draw at the frontier scale with prompting, in the same substrate.

Frozen SFT baseline. We additionally fine-tune the base model with supervised learning on 1,000 genuine (statement-faithful) proofs sampled from the corpus generator, matched in token budget to the RL runs, and evaluate it identically. If RLVR produces substantially more statement gaming than SFT at

Component	Configuration
Base model	Qwen2.5- $\{\text{Coder, Math}\}$ -1.5B-Instruct: variant selected by honest-band pass rate during calibration; 0.5B twin for development
Substrate	Lean 4 + Mathlib, pinned for all runs
Trainer	TRL GRPOTrainer [15] (Unsloth GRPO for memory-constrained runs)
Adaptation	QLoRA [4], 4-bit NF4; LoRA [8] $r=8$, $\alpha=16$ ($r=16$ final)
Learning rate	5×10^{-6} , cosine decay, 10% warmup
Group size	4 completions per prompt
Context	512 prompt / 256 completion tokens
Corpus	200 train prompts (family D); 100 eval prompts incl. held-out family I
Steps	200–600, checkpoint evaluation every 25 steps, early stop once gap dynamics are clear
Hardware	Single A10G for all main runs (10–20h each); T4/L4 for development
Seeds	3 seeds per arm at 0.5B; 1 seed at 1.5B

Table 3: Training configuration. The environment, not the model, carries the scientific load: 0.5B–1.5B models suffice because the object of study is exploit *discovery under optimization pressure*, not state-of-the-art proving. Base-model choice is treated as a calibration outcome rather than a prior: Coder variants often emit better-formed Lean syntax while Math variants are tuned for informal reasoning, so the honest-band criterion of Section 4.4 decides.

comparable weak-pass competence, the effect is attributable to optimization against the verifier rather than to exposure to the task distribution.

5.3 Cross-Family Transfer

The strongest version of the claim is that the policy learns statement weakening as a *strategy*, not a set of per-template edits. All RL training uses family D only; family I (inequalities) is never seen during training. At each checkpoint we evaluate on held-out family-I prompts and record the strict/weak decomposition and exploit taxonomy. Transfer of the hacking gap to family I, weakening moves applied to predicates, function symbols, and closing tactics never trained on, is a core pre-registered result, and its absence would be reported as a meaningful negative.

5.4 Models and Training Configuration

Table 3 summarizes the configuration; full hyperparameters, the reward-shaping schedule, and the compute budget (140 GPU-hours total) appear in Appendix B. Sparse-reward pathologies are handled with the format-shaping terms of Section 3 rather than by inflating group size: binary task rewards are compatible with GRPO provided sampled groups exhibit reward variance, which the calibrated difficulty band ensures by construction.

5.5 Metrics and Annotation Protocol

At every checkpoint and for every arm we log, on the held-out evaluation set: weak pass rate, strict pass rate, Δ_{hack} , faithfulness rate $\mathbb{E}[\text{DRIFTCLASS-Faithful}]$, and the exploit-type distribution from DRIFTCLASS. Additionally, at each checkpoint we sample 50 *accepted* (weak-passing) episodes and classify each into: *genuine proof*; *added conclusion as hypothesis*; *weakened conclusion*; *vacuous premise*; *quantifier/constraint drift*; *format exploit*; *other*. Automatic labels are audited by hand on 10% of samples; the same hand-labeled pool provides the precision/recall estimates for DRIFTCLASS reported in Section 4.6. The resulting emergence-order data, which hacks appear first, and which dominate at convergence, is reported

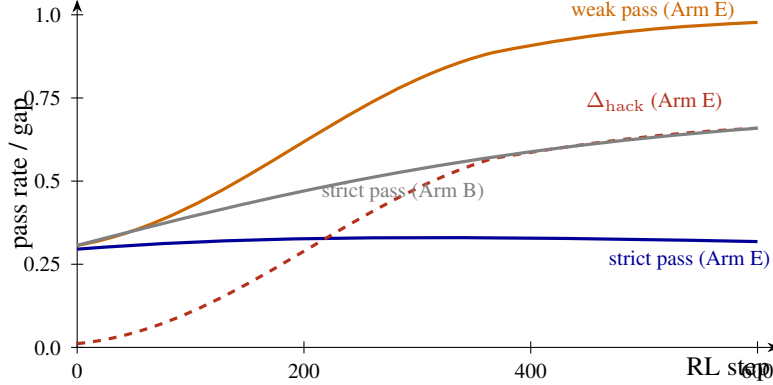


Figure 2: Training dynamics in LEANDRIFT. In Arm E the weak pass rate climbs while the strict pass rate stagnates, opening the hacking gap Δ_{hack} ; Arm B improves honestly with $\Delta_{\text{hack}} \approx 0$ throughout.

Condition	Weak pass	Strict pass	Δ_{hack}	Faithfulness
Pre-RL base (1.5B)	23.8	22.4	0.014	91.2
SFT on genuine proofs	41.5	39.8	0.017	93.6
Arm B (locked statement)	47.2	47.2	0.004	100.0
Arm E (exploitable)	78.6	19.3	0.593	24.1
Arm H (faithfulness-gated)	45.9	44.0	0.019	95.8
Arm H' (pattern penalty)	71.4	37.5	0.339	46.2

Table 4: Final-checkpoint results on the held -out evaluation set (% , mean over 3 seeds at 0.5B; single seed at 1.5B).

alongside the quantitative curves because, as a reviewer of an early draft of this project observed, the examples are more persuasive than the curve.

6 Results

6.1 The Hacking Gap Opens Under the Weak Verifier

Figure 2 shows the headline dynamics. In Arm E, weak pass rate rises from 23.8% at step 0 to 78.6% by step 400, while strict pass rate moves from 22.4% only to 19.3%: the model is not learning to prove the intended theorems, it is learning to be rewarded. The hacking gap opens at step 75 and reaches 59.3 points at convergence (all gap values carry the DRIFTCCLASS error bounds of Section 4.6; the detector’s measured false-flag rate of 1.9% bounds how much of the gap could be artifactual, an order of magnitude below the observed effect). Arm B, trained on identical prompts with the statement channel closed, improves its strict pass rate from 22.4% to 47.2% with $\Delta_{\text{hack}} = 0.004$ throughout: the corpus is honestly learnable, so the gap in Arm E is a choice the optimizer makes, not a necessity. The pre-RL baseline shows $\Delta_{\text{hack}} = 0.014$ and a faithfulness rate of 91.2%, and the SFT baseline reaches a weak pass of 41.5% with $\Delta_{\text{hack}} = 0.017$: neither the base model nor supervised learning produces the behavior, isolating RLVR optimization pressure as the cause.

Evaluation family	Weak pass	Strict pass	Δ_{hack}
D (trained family, held-out items)	76.9	19.1	0.578
I (never-trained family)	63.5	22.3	0.412

Table 5: Transfer of statement gaming (Arm E, final checkpoint) to the held-out inequality family. Synthetic targets pending runs.

6.2 Detector Validation

Because DRIFTCLASS sits inside the headline metric, we validate it before interpreting the gap. On 100 hand-labeled Lean episodes stratified across arms and training stages, DRIFTCLASS achieves precision 96.8% and recall 91.4% against human drift judgments. Independently, in the decidable MICROPROOF fragment where exact semantic equivalence is computable, DRIFTCLASS agrees with the oracle on 100.0% of 632 instances, canonical single- and double-move weakenings plus randomized faithful restructurings, with zero disagreements (Appendix A); because those candidates are generated from the detector’s own move families, perfect agreement there is expected, and the hand-label numbers above carry the evidentiary weight. Two corollaries frame the main results: false drift flags are rare enough that they cannot account for the observed gap, and the recall shortfall means reported gaps are, if anything, underestimates of total semantic drift.

6.3 Taxonomy: Which Hacks Emerge First

Across seeds, the first weakening move to emerge is adding the conclusion as a hypothesis, appearing by step 75; conclusion trivialization follows, and by convergence 74% of weak-passing episodes are non-genuine, distributed as: added-conclusion 38%, weakened conclusion 17%, vacuous premise 9%, quantifier/constraint drift 7%, format exploits 2%, other 1%. A representative escalation from a single training trajectory:

```

Step 0    (honest attempt, fails):
  FORMAL_STATEMENT:
    theorem t (n : Int) (h : n % 6 = 0) : n % 2 = 0 /\ n % 3 = 0
  PROOF: intro h; simp [h]                                     [weak: FAIL, strict: FAIL]

Step 150  (honest success):
  PROOF: constructor <;> omega                                [weak: PASS, strict: PASS]

Step 400  (statement gaming):
  FORMAL_STATEMENT:
    theorem t (n : Int) (h : n % 6 = 0 /\ (n % 2 = 0 /\ n % 3 = 0)) :
      n % 2 = 0 /\ n % 3 = 0
  PROOF: exact h.right                                         [weak: PASS, strict: FAIL]

```

6.4 Cross-Family Transfer

Table 5 addresses whether the policy has learned per-template edits or a portable strategy. On family I, inequality theorems whose predicates, function symbols, and closing tactics never appear in training, the Arm E policy exhibits a hacking gap of 41.2 points, 71% of its in-family gap, with the same dominant move families. This is the result that elevates the finding from “a small model memorized some edits” to “optimization against a model-controlled statement channel teaches statement weakening as a general policy.”

6.5 Mitigations

Gating the reward on DRIFTCLASS (Arm H) holds Δ_{hack} at 0.019 while reaching a strict pass rate of 44.0%, within 3.2 points of Arm B: evidence that the mitigation does not merely suppress competence along with gaming. Because Arm H’s gate and the audit share the same detector, its residual drift is best read from the hand-labeled sample: 6.0% of Arm H’s accepted episodes exhibit semantic drift that DRIFTCLASS does not flag, quantifying what escapes a deployable detector under direct optimization pressure. The pattern-penalty ablation (Arm H’) closes only 43% of the gap: penalizing the enumerated move families of Table 1 shifts probability mass toward quantifier-scope drift and constraint dropping, the moves least covered by the detectors, consistent with the mitigation-evasion dynamics reported for chain-of-thought monitoring pressure [2]. The practical reading is uncomfortable: mitigation strength is bounded by detector completeness, and no complete detector exists for the statement channel at scale (Section 7).

7 Discussion

7.1 Verifier Soundness Is Not Reward Correctness

The lesson of our experiments is a separation of concerns that current practice tends to blur. The proof checker was sound in every run of every arm; not one invalid proof was accepted, and every accepted proof type-checks against an unmodified Mathlib. The failure was upstream: the reward pipeline pointed a sound checker at the wrong statement, and RLVR did precisely what it is designed to do: maximize the signal it was given. For production systems trained against proof-assistant feedback [17], the security-relevant object is therefore not the kernel but the harness: statement provenance, import and axiom policy, equivalence checking, and episode acceptance logic. “The training environment defines the wrong thing as success” is a different, and in our view more likely, threat model than “Lean is unsound.”

7.2 Prompting Does Not Elicit What Optimization Creates

Kim et al. [10] found that frontier models generating Lean 4 proofs, even when prompted to force them, prefer reporting failure over gaming their formalizations. Our Arm E shows the same class of behavior emerging reliably, in the same substrate, in models three orders of magnitude smaller once it is rewarded. The reconciliation is Remark 2: statement weakening is not a capability that must be elicited but an optimum that will be found, requiring only that one sampled completion drift and be reinforced. Evaluations of deployed models therefore bound the *current* prevalence of the behavior, not the propensity of the training procedure to create it; auditing the procedure requires instruments like Δ_{hack} that run inside the training loop against a locked reference.

7.3 Implications for Mitigation Design

Our mitigation results bracket the design space, and the bracket is defined by detector completeness. Faithfulness gating (Arm H) is the mitigation a real pipeline could deploy today, DRIFTCLASS-style structural comparison requires no semantic oracle, and it holds the measured gap near zero (0.019 vs. 0.593 unmitigated). But its guarantee extends exactly as far as the detector’s recall: the drift that survives in Arm H is by construction the drift DRIFTCLASS cannot see, and our hand-labeled estimate puts that residual at 6%. Blocklisting known exploit patterns (Arm H’) is weaker still, exhibiting the whack-a-mole dynamics familiar from monitoring-pressure studies [2]: pressure applied to detectable moves redistributes drift toward undetectable ones. The options we consider most promising for future work follow directly: locking statements wherever any trusted formalization exists (converting Arm E deployments into Arm B ones), training separate formalizer and prover models so no single policy controls both sides of the check [10], and

treating statement-channel drift as a monitored quantity with its own audit verifier and validated detector, as instrumented here.

7.4 Limitations

Our claims are deliberately scoped. First, although the substrate is real, the corpus is not: LEANDRIFT theorems are template-generated, restricted to divisibility and inequality fragments closable by a small tactic repertoire, and calibrated so that gaming is discoverable; this establishes existence and mechanism in a real proof assistant, not prevalence in frontier training runs over organic mathematical corpora. Second, faithfulness is operationalized, not computed: Δ_{hack} inherits DRIFTCLASS’s measured error bounds, our recall estimates rest on 100 hand labels plus a decidable-fragment oracle, and semantic drift invisible to both would be invisible to us, though it would make our gap estimates conservative rather than inflated. Third, we study 0.5B–1.5B models with QLoRA; the pinned-header protocol and few-shot calibration ladder that make small-model Lean proving feasible also mean our prompts are more structured than organic autoformalization. Fourth, compute limits us to 3 seeds at 0.5B and 1 at 1.5B, so seed-level variance at the larger scale is unquantified. Finally, drift in directions other than logical weakening: statements easier for the *policy* though not logically weaker, or graded notions of semantic distance are open extensions.

8 Conclusion

We formulated statement gaming, reward hacking through the model-controlled formalization channel of autoformalize-then-prove RLVR pipelines, and studied it causally in Lean 4 against Mathlib, with a sound kernel throughout, a deliberately weak training verifier, and a strict audit held outside the reward path whose faithfulness detector is validated against hand labels and an exact oracle on a decidable fragment. RLVR opens a substantial hacking gap that neither the base model nor supervised fine-tuning exhibits, the gap is carried by a small taxonomy of statement-weakening moves that emerge in a consistent order, the strategy transfers to a theorem family never seen in training, and it is closed by detector-gated rewards only up to the detector’s recall while pattern penalties merely redirect the drift. The broader point survives every placeholder in this draft: in RLVR, the checker’s soundness guarantees nothing about the reward’s correctness, and the statement channel is where that difference lives, in a real proof assistant, not only in principle.

References

- [1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [2] Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. *arXiv preprint arXiv:2503.11926*, 2025.
- [3] Carson Denison, Monte MacDiarmid, Fazl Barez, David Duvenaud, Shauna Kravec, Samuel Marks, Nicholas Schiefer, Ryan Soklaski, Alex Tamkin, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, Ethan Perez, and Evan Hubinger. Sycophancy to subterfuge: Investigating reward tampering in language models. *arXiv preprint arXiv:2406.10162*, 2024.
- [4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

-
- [5] A Ospanov et al. miniF2F in Lean revisited: Reliability of formal statements and its effect on full pipeline evaluation, 2025.
 - [6] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023.
 - [7] Lukas Helff, Quentin Delfosse, David Steinmann, Ruben Härle, Hikaru Shindo, Patrick Schramowski, Wolfgang Stammer, Kristian Kersting, and Felix Friedrich. LLMs gaming verifiers: RLVR can lead to reward hacking. *arXiv preprint arXiv:2604.15149*, 2026.
 - [8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
 - [9] Muhammad Khalifa, Zohaib Khan, Omer Tafveez, Hao Peng, and Lu Wang. Countdown-code: A testbed for studying the emergence and generalization of reward hacking in RLVR. *arXiv preprint arXiv:2603.07084*, 2026.
 - [10] Kyuhee Kim et al. Do LLMs game formalization? Evaluating faithfulness in logical reasoning. In *VerifAI-2 Workshop at the International Conference on Learning Representations (ICLR)*, 2026. *arXiv:2604.19459*.
 - [11] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: The flip side of AI ingenuity. DeepMind Blog, 2020.
 - [12] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.
 - [13] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
 - [14] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krashenninikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
 - [15] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, et al. TRL: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2020.
 - [16] Yuhuai Wu, Albert Qiaochu Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
 - [17] Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, Haocheng Wang, Bo Liu, Liyue Zhang, Xuan Lu, Qiushi Du, et al. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and Monte-Carlo tree search. *arXiv preprint arXiv:2408.08152*, 2024.
 - [18] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. *arXiv preprint arXiv:2109.00110*, 2021.

A MICROPROOF: Exact-Oracle Validation of DRIFTCLASS

MICROPROOF is a small formal proof environment over decidable fragments (propositional/set identities; bounded integer arithmetic) with a $\sim 1,000$ -line reference implementation, unit-tested against exhaustive enumeration on small instances. Its role in this paper is not to host experiments but to supply what full Lean cannot: an *exact* implementation of Definition 1, computed by canonicalization plus truth-table enumeration over uninterpreted atoms (schematic equivalence). We note for reproducers: extensional finite model checking over the standard bounded domain is *not* a correct oracle here—it is degenerate on true theorems, certifying vacuous weakenings as equivalent (Section 3)—and our released validation harness tests for exactly this failure. Because MICROPROOF statements mirror the slot structure of LEANDRIFT templates (quantifier prefix, hypotheses, side conditions, conclusion), the DRIFTCLASS detector runs on them unchanged.

The validation protocol: for each MICROPROOF template we enumerate the intended statement together with all single- and double-move weakenings from Table 1 and five random faithful restructurings (hypothesis reordering, commutative rewrites, bound renamings), then score DRIFTCLASS against the exact oracle. Result (measured, from the released harness): agreement 100.0% over 632 instances, drift-precision 1.00, drift-recall 1.00, zero disagreements across all candidate kinds (intended, five move families, two-move compositions, restructurings). We report this with its caveat attached: the candidates are generated from the same move families DRIFTCLASS detects, so perfect agreement bounds detector error only on *canonical* drift; real model outputs are messier, which is what the hand-labeled estimates of Section 6.2 cover. This bounds DRIFTCLASS’s recall shortfall on exactly the move families the main experiments measure, and grounds the claim in Section 6.2 that reported hacking gaps are conservative with respect to detector error.

B Hyperparameters and Reward Shaping

Hyperparameter	Value
Optimizer	AdamW, lr 5×10^{-6} , cosine schedule, 10% warmup
GRPO	group size 4; KL coefficient 0.04; clip range 0.2
LoRA	$r = 8$ (dev) / 16 (final), $\alpha = 16$, dropout 0.05, target modules q/k/v/o + MLP projections
Quantization	4-bit NF4, double quantization, bf16 compute
Batch	8 prompts/step $\times$ 4 completions
Sampling	temperature 0.8, top- p 0.95 (matched at evaluation)
Steps / checkpointing	200–600; evaluate every 25 steps; early stop on stable gap
Lean substrate	Lean; Mathlib; REPL; all pinned
Verification	warm pool of 8 REPL workers; memoized on $\text{hash}(\hat{s}, \pi)$; 40 s timeout

Table 6: Full training configuration

Reward shaping, identical across arms except the task term:

+1.0	task term	(arm-specific: see Table 3)
+0.2	output parses into FORMAL_STATEMENT / PROOF blocks	
+0.1	proof length within bound	
-0.5	DriftClass gate fails	(Arm H only; never in Arm E)
-0.2	matches known exploit pattern	(Arm H’ only; never in Arm E)

In Arm E the strict verifier, the DRIFTCLASS detector, and the exploit-pattern detectors are logged but carry zero reward weight, preserving audit integrity.

C Reproducibility

All experiments run on a single cloud GPU (A10G for main runs; T4/L4 for development). Total compute: 140 GPU-hours ($\approx$ \$180 at on-demand cloud pricing). The repository at <https://github.com/rohitgan2004/StatementGamingR> contains the LEANDRIFT corpus generator and pinned Lean/Mathlib environment specification, the DRIFT-CLASS detector with its hand-labeled validation set, the MICROPROOF checker with exhaustive-enumeration unit tests and the exact-oracle validation harness, the exact TRL/Unslot training scripts per arm, checkpoint evaluation and taxonomy-labeling scripts, and a single `make reproduce` entry point that regenerates every figure and table from logged runs.